

100 Days of Azure DevOps

Day 84 Handout — Migrate the guarantees, leave the nostalgia

9 - Advanced & Enterprise · Personal lab only · Educational content · Not a sales pitch

Architecture A — Jenkinsfile concepts → Azure YAML

Jenkins	Azure Pipelines	Guarantee to keep
agent { label 'linux' }	pool: vmImage: ubuntu-latest (or a named pool)	Same OS / network constraint, not the same snowflake
stages { stage('Build') }	stages / jobs / steps	Order and failure isolation
post { always { } }	condition: always() on a job or a later stage	Cleanup / publish still runs
credentials('id')	Variable group or Azure Key Vault task	Secret not in SCM
stash / artifact	PublishPipelineArtifact@1 + download	Same bytes promoted later
Jenkins plugin X	Built-in task, script, or a written gap	Do not hide a plugin as 'YAML not ready'

Architecture B — migrate vs leave on paper

Piece	Migrate today (paper)	Leave / rewrite
Compile + test + publish	Map to jobs on ubuntu-latest	Pixel-identical stage names
Shared library (vars/)	Template repo or templates/	Groovy DSL as a religion
Freestyle click jobs	Rewrite as YAML; do not clone the UI	Rebuilding Jenkins screens in ADO
On-prem agent label	Named pool only if a real constraint	'We need the same snowflake' nostalgia
Plugin that was the product	Write the gap in the mapping table	Pretend Azure has a 1:1 task

Architecture C — pitfalls that stall a migration

Pitfall	What you see	Fix
Nostalgia migration	Blue-ball UI rebuilt in Boards	Map guarantees; rename stages freely
Secret lift-and-shift	credentials copied into YAML	Variable group / Key Vault from Phase 7
Plugin iceberg	'YAML not ready' forever	List plugins; each is a negotiation
Standing up Jenkins for the lab	You install Jenkins to 'be fair'	A 20-line sample Jenkinsfile on paper is enough
Employer Jenkinsfile paste	Internal job names in the post	Write a tiny sample; no work SCM

One-liner to remember

Jenkins migrations succeed when you migrate pipelines, not nostalgia.

Step-by-step lab (personal Azure / Azure DevOps only)

Paper lab on the personal repo. Do not install Jenkins. Do not paste an employer Jenkinsfile.

- 1 Write samples/jenkins/Jenkinsfile with agent, three stages (Build, Test, Publish), and a post { always } block. Keep it tiny and fictional.
- 2 Create docs/jenkins-map-day84.md with a table: Jenkins line → Azure YAML → guarantee kept.
- 3 Translate to samples/jenkins/azure-pipelines.yml: pool ubuntu-latest, stages/jobs, condition: always() publish, no inline secrets.

- 4 Map credentials('id') to a variable group name or Key Vault task — write the name, do not put a value in Git.
- 5 List two Jenkins plugins the sample does not need, and one fictional plugin that would be a real gap (e.g. a licensed scanner).
- 6 Do not stand up a Jenkins controller. The proof is the mapping, not a blue ball.

Same guarantees — Jenkinsfile fragment vs Azure YAML

```
# Jenkins (sample only)
# agent { label 'linux' }
# stage('Test') { sh 'dotnet test' }
# post { always { archiveArtifacts '**/TestResults/**' } }

pool:
  vmImage: ubuntu-latest
stages:
- stage: Test
  jobs:
  - job: test
    steps:
    - script: dotnet test --logger trx
      displayName: Test
    - task: PublishTestResults@2
      condition: always()
    inputs:
      testResultsFormat: VSTest
      testResultsFiles: '**/*.trx'
```

Done checklist

- Sample Jenkinsfile and mapped azure-pipelines.yml both exist
- docs/jenkins-map-day84.md lists agent/stages/post/credentials/plugins
- No employer Jenkinsfile
- No Jenkins controller installed for the lab
- Posted the LinkedIn document

Tomorrow — Day 85

Hybrid and multi-cloud CI/CD — survey the risks; do not buy a second cloud for a slide